

☑ UNIT-II : Advanced Data Structures – Complete Notes

(Covers: Red-Black Trees, B-Trees, Graphs, BFS/DFS, Disjoint Sets)

-

1. RED-BLACK TREES

-

✓ What is a Red-Black Tree?

A **Red-Black Tree (RBT)** is a **self-balancing Binary Search Tree (BST)** where each node has an extra color attribute **RED** or **BLACK**.

Its purpose:

- 👉 Maintain height = $O(\log n)$
- 👉 Ensure fast search, insert, delete

✓ Properties of Red-Black Trees

Every RBT must satisfy:

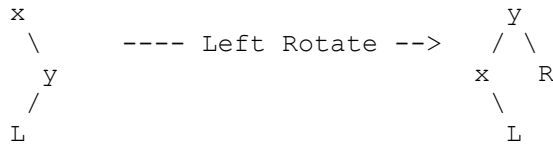
1. **Each node is either RED or BLACK.**
2. **Root is always BLACK.**
3. **All leaves (NULL children) are BLACK.**
4. **RED node cannot have a RED parent.**
5. **Every path from a node to its descendant NULL nodes must have the same number of BLACK nodes.**
(Called *Black Height*)

These rules ensure balancing.

✓ Rotations (Left & Right Rotations)

Used to maintain the BST balance.

Left Rotation (rotate_left(x))



Algorithm: LEFT-ROTATE(x)

```
y = x.right
x.right = y.left
if (y.left != NULL)
    y.left.parent = x
y.parent = x.parent
if (x.parent == NULL)
    root = y
else if (x == x.parent.left)
    x.parent.left = y
else
    x.parent.right = y
y.left = x
x.parent = y
```

Right Rotation is symmetrical.

✓ Insertion in Red-Black Tree

Steps:

1. Insert node as BST (color = RED)
2. Fix violations using:

- Recoloring
- Rotations (LL, RR, LR, RL)

Cases to Fix Red-Red Violation

1. **Uncle is RED** → recolor
2. **Uncle is BLACK** → rotation + recolor

✓ Algorithm: RB-INSERT(T, z)

```

1. Insert z into tree T as in BST (color z = RED)
2. While (z.parent.color == RED):
    If z.parent is left child:
        Handle 3 cases (uncle cases)
    Else:
        Mirror cases
3. root.color = BLACK

```

✓ Deletion in Red-Black Trees

Steps:

1. Perform BST deletion
2. If deleting BLACK node → may violate black height
3. Fix using:
 - Recoloring
 - Rotations

Algorithm: RB-DELETE(T, z)

(High-level)

```

1. y = z
2. Save y's original color
3. Replace z with successor (if needed)
4. If original color was BLACK → call RB-DELETE-FIXUP()

```

Fixup handles cases of "double black".

2. B-TREES

✓ What is a B-Tree?

A **multi-way self-balancing search tree** used in:

- Databases
- File systems
(because it reduces disk accesses)

A B-Tree of **order m**:

- Each node can have **at most m children**
 - Each node has **(m-1)** keys
 - All leaves appear at the **same depth**
-

✓ Properties of B-Tree

1. Root has at least **1** key.
 2. Every internal node has $\lceil m/2 \rceil$ to m children.
 3. Keys are sorted.
 4. Leaves appear at same level.
-
-

✓ Basic Operations

Search

Same as multi-way BST search.

Algorithm: B-TREE-SEARCH(x, k)

```
i = 1
while i ≤ n[x] and k > key[i]:
    i = i + 1

if i ≤ n[x] and k == key[i]:
    return (x, i)
else if leaf[x] == TRUE:
    return NOT FOUND
else:
    return B-TREE-SEARCH(child[i], k)
```

✓ Insertion

Steps:

1. Search proper leaf.
2. If node is not full → insert.
3. If node is full → split before descent.

Algorithm: B-TREE-INSERT(T, k)

```
If root is full:
    s = new node
    root = s
    s.child[1] = old root
    SPLIT_CHILD(s, 1)
InsertNonFull(s, k)
Else:
    InsertNonFull(root, k)
```

SPLIT CHILD

Splits a full node into 2 half nodes.

✓ Deleting a Key in B-Tree

Cases:

1. Delete from leaf
2. Delete from internal node
 - Replace by predecessor/successor
3. Ensure nodes have minimum occupancy
 - Borrow from sibling
 - Merge siblings

Algorithm: B-TREE-DELETE(T, k)

(High-level)

1. Find key
 2. If in leaf → remove directly
 3. If in internal:
 - Replace by predecessor OR successor
 4. Before recursion, ensure child has $\geq t$ keys
 5. Handle merge/borrow cases
-

-

3. GRAPHS

-

✓ Graph Representation Methods

Two main types:

✓ 1. Adjacency Matrix

- 2D matrix of size $n \times n$
- $\text{graph}[i][j] = 1$ if edge exists else 0

Example

```
0 1 0
1 0 1
0 1 0
```

✓ 2. Adjacency List

- Each vertex stores a linked list of neighbors
- Saves memory

Example

```
0 → 1
1 → 0 → 2
2 → 1
```

-

4. GRAPH TRAVERSALS

-

✓ 1. Breadth-First Search (BFS)

Uses Queue – explores level by level.

Algorithm: BFS(G, s)

```
for each vertex v:
    color[v] = WHITE
```

```

        dist[v] =  $\infty$ 

color[s] = GRAY
dist[s] = 0
enqueue(s)

while queue not empty:
    u = dequeue()
    for each v in Adj[u]:
        if color[v] == WHITE:
            color[v] = GRAY
            dist[v] = dist[u] + 1
            enqueue(v)
    color[u] = BLACK

```

✓ 2. Depth-First Search (DFS)

Uses Stack / Recursion – explores deep paths first.

Algorithm: DFS(G)

```

for each vertex u:
    color[u] = WHITE

for each vertex u:
    if color[u] == WHITE:
        DFS-VISIT(u)

```

DFS-VISIT(u)

```

color[u] = GRAY
for each v in Adj[u]:
    if color[v] == WHITE:
        DFS-VISIT(v)
color[u] = BLACK

```

-

5. DISJOINT SET DATA STRUCTURE (Union–Find)

Used in:

- Kruskal's Algorithm
- Network connectivity
- Dynamic grouping

✓ Operations

1. **MAKE-SET(x)** → create a new set containing x
 2. **FIND-SET(x)** → find representative (root)
 3. **UNION(x, y)** → merge two sets
-

✓ Representations

1. Forest Representation (Trees)

Each set = one tree

- Root = representative
 - Path compression reduces height
-

✓ Algorithms

MAKE-SET(x)

```
parent[x] = x  
rank[x] = 0
```

FIND-SET(x) with Path Compression

```
if x != parent[x]:
    parent[x] = FIND-SET(parent[x])
return parent[x]
```

UNION(x, y) – By Rank

```
xRoot = FIND-SET(x)
yRoot = FIND-SET(y)

if xRoot == yRoot:
    return

if rank[xRoot] < rank[yRoot]:
    parent[xRoot] = yRoot
else if rank[xRoot] > rank[yRoot]:
    parent[yRoot] = xRoot
else:
    parent[yRoot] = xRoot
    rank[xRoot] = rank[xRoot] + 1
```

-

✓ Summary Table

Topic	Key Data Structures	Operations	Time Complexity
Red-Black Tree	Self-balancing BST	Rotate, Insert, Delete	$O(\log n)$
B-Tree	Multi-way tree	Search, Insert, Delete	$O(\log n)$
Graphs	Adj List/Matrix	BFS, DFS	$O(V+E)$
Disjoint Sets	Union-Find	Find, Union	Near $O(1)$

-